



Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Concepts and Characteristics of OOP

OOP



Learning Outcome:

Describe common activities at work to a visitor using short sentences in familiar spoken and written situations.

Explain important features related to object programming in different contexts and with different types of object programming.

Ask your partner:

- How did programming languages work in the past?
- What object based languages do you know? What is your favorite? Why?

Can Do Statements:

- I can explain what OOP is.
- I can give information about key features related to OOP.
- I can talk briefly about the history of OOP.





Let's Explore!

The Evolution of OOP



Instructions: Read the text below.

Mike: Hi, Lui. What are you doing?

Lui: I'm reading about the history of OOP.

Mike: What does it say?

Lui: The author is telling that Object-Oriented Programming or OOP was the new conception of programming in the late 70's; it's a programming method that radically changed the way how programming worked. Its approach, based on the idea that we interact with single objects, it becomes part of a whole, so we cannot mix their attributes and capabilities. Therefore, each piece must have its own independent features.

Mike: Did programming perform differently in the past?

Lui: Yes. The earliest approaches that programming managed, consisted of an arrangement of variables and data to be included into an algorithm to fulfill the desired results. This was useful for simple programs, but as the complexity increased, it became more problematic to apply this method for any modification, required in its original definitions of data structured, it could completely disturb the program running in other stages. Computer programmers tried to establish some programming rules to avoid side effects, but their new ways to program were still in process.

Mike: What are examples of OOP languages?

Lui: The first OOP language was Simula 67, created in the late 60's; Smalltalk created in the 70's; finally, C++ arrived in the 80's. C++ is probably the most used programming language, and it holds all the features of an OOP language. Nowadays, languages such as Java and Visual Basic are becoming very popular in OOP languages.

Mike: What are the main features of OOP?

Lui: The main feature is the programming itself, because it is made with objects, and each object is independent. So, it has its own characteristics and procedures which indicate that it is an innovative way to integrate data.





Let's Practice!

Let's look back* to the OOP's history!

*"Look back" refers to remember something that happened in the past.

Instructions: Match the sentences from Column A with Column B.

COLUMN A

1. Object-Oriented Programming or
2. Its approach, based on the idea that we interact with single objects,
3. People is emphasizing the use of OOP

COLUMN B

- a. it becomes part of a whole, so we cannot mix their attributes and capabilities.
- b. OOP was the new conception of programming in the late 70's.
- c. as a programming method to interact with objects.



Let's Practice!

OOP History in Brief

Instructions: Organize the mystery sentence.

1. was / OOP / 67. / first / Simula / The / language

2. to / 70's. / the / came / Smalltalk / Then, / life / in

3. 3. in / 80's. / Finally, / C++ / arrived / the



Extension Activity

OOP's History, Concepts, and Characteristics

Instructions: Complete the following steps. Fill in the Self-Assessment.

- List 10 facts about OOP in history.
- Create a comparative table that represents OOP in the past, in the present, and nowadays.
- Describe specific OOP features in the different timelines.

Self-Assessment

Yes

No

- I identify OOP in history.
- I compare OOP in different timelines.
- I describe OOP features past and present.

☐	☐
☐	☐
☐	☐



Target Words

- **fulfill** (verb): to satisfy
e.g. The earliest approaches that programming managed, consisted of an arrangement of variables and data to be included into an algorithm to fulfill the desired results.
- **avoid** (verb): to evade
e.g. Computer programmers tried to establish some programming rules to avoid side effects.
- **side effect** (compound noun):
reaction, symptom
e.g. Computer programmers tried to establish some programming rules to avoid side effects, but their new ways to program were still in process.
- **capability** (noun): capacity, skill
e.g. Its approach becomes part of a whole, so we cannot mix their attributes and capabilities.
- **whole** (noun): the total amount of something
e.g. Its approach, based on the idea that we interact with single objects, it becomes part of a whole.
- **own** (adjective): personal, that belongs to oneself
e.g. Therefore, each piece must have its own independent features.



Learning Points

Simple Past, Simple Present, and Present Continuous

You can use these structures to talk about the development of object oriented programming through history. We are going to review the **simple past**, the **simple present**, and the **present continuous**.

Simple past is the tense you use to talk about past events, therefore use it when you want to talk about the history of things. Let's take a quick look at the chart which contains the structures and examples in *affirmative*, *negative*, and *simple questions*.

Affirmative	Negative	Simple Questions
Structure: Subject + verb in simple past + complement.	Structure: Subject + did not /didn't + verb in present simple + complement.	Structure: Did + subject + verb in simple present + complement + ?
Example: The first programmers developed Simula 67, the first object-oriented language, in the 1960's.	Example: The first programmers didn't develop Simula 67, the first object-oriented language, in the 1960's.	Example: Did the programmers develop Simula 67, the first object-oriented language, in the 1960's?

Remember, there are two main forms of the verbs in simple past, **regular verbs** and **irregular verbs**. You can find lists in books and on the Internet.

Simple present is the tense you use to talk about facts, so use it when you want to talk about general facts about something. Let's take a quick look at the chart which contains the structures and examples in *affirmative*, *negative*, and *simple questions*.





Learning Points

Affirmative	Negative	Simple Questions
Structure Subject + verb in present simple + complement Note: Is it called third person rule, if the sentence use the third person singular, you add -s, -es, -ies to the verbs. This rule doesn't apply with the present of the verb to be.	Structure: Subject + do not or don't/ does not or doesn't + complement. Remember, if you have the auxiliary does in the sentence, the third person rule doesn't apply.	Structure: Do or Does + subject + verb in simple present + complement + ? Remember, if you have the auxiliary does in the question, the third person rule doesn't apply.
Examples: OOP possesses many different features.	Examples: OOP doesn't possess many different features.	Examples: Does OOP possess many different features?

Present continuous is the tense you use to talk about actions in progress, therefore use it when describing events of something happening at the moment. Let's take a quick look at the chart which contains the structures and examples in *affirmative*, *negative*, and *simple questions*.

Affirmative	Negative	Simple Questions
Structure: Subject + am/is/are + verb + ing + complement.	Structure: Subject + am not/ is not/ are not + verb + ing + complement.	Structure: Is/Are + subject + verb + ing + complement + ?
Example: People are learning more about object-orienting languages.	Example: People aren't learning more about object-orienting languages.	Example: Are people learning more about object-orienting languages?





Learning Points

Examples

- *I'm reading about the history of OOP.*
- *The author is telling that Object-Oriented Programming or OOP was the new conception of programming in the late 70's.*
- *Its approach, based on the idea that we interact with single objects, it becomes part of a whole, so we cannot mix their attributes and capabilities.*
- *The earliest approaches that programming managed, consisted of an arrangement of variables and data to be included into an algorithm to fulfill the desired results.*

Possessive Adjectives: Its - Theirs

Possessive adjectives are words you use to point out the possession in a sentence.

You use **its** to indicate that an object or set of objects refers to something.

Example:

*This is first **object**, **its** background color is blue.*

You use **their** to indicate that an object or set of objects refers to several things or more than one person.

Example:

*Those are **the new programmers**, Tom and Clarice, **their** job is easier thanks to OOP.*

Note: You choose to use **its** or **their** depending on the owner, not on the objects.

Examples:

Its (the object) color is red.

Their (Tom and Clarice) job is easier thanks to OOP.

Examples

- *Therefore, each piece must have its own independent features.*
- *This was useful for simple programs, but as the complexity increased, it became more problematic to apply this method for any modification, required in its original definitions of data structured.*
- *Computer programmers tried to establish some programming rules to avoid side effects, but their new ways to program were still in process.*

